

先看懂：哪些 token 要重新计算？

从一个 10,000 token 的 prompt 开始。把 token 理解为模型处理文本的小单位即可。

你说的 miss：prefill 阶段，无法复用已有 KV、需要重新计算的 token 数量或比例。这里按“可复用前缀 + 新计算后缀”理解。[1][2]

教学例子：总长 10,000 token，miss 比例 20%



命中缓存，也可能需要读盘。命中表示“有可复用结果”；结果是否已经在 NPU 内存里，是另一个问题。

KV 是可复用的中间计算结果。NPU 负责计算；SSU 是存放 KV 的存储设备；I/O 是搬运数据。prefill 是处理输入、为生成答案做准备的阶段。

后面都用：L = 总 token 数；p = miss 比例；M = miss token 数。

$M = p \times L = NQL$ 。例如 $p = 20\% = 0.2$ 。

边界：若命中 KV 已驻留 NPU 内存，就不需要全部从 SSU 读取；本图不能直接套用。来源 [1][2] 见第 11 页。

为什么 miss 越少，搬运反而越吃紧？

比较同样长度的输入：少算了很多，仍要搬运大部分已有 KV。

固定总长和执行配置时，可先近似理解为：
计算时间 $C \approx \text{每 token 计算成本} \times \text{miss token 数}$
读取量 $D = \text{每 token 的 KV 大小} \times \text{命中 token 数}$

这里 p 是 miss 比例，20% 写成 0.2； D 表示读取的数据量。要在计算期间读完，需要的平均速率约为 $B = D / C$ 。每 token 计算成本固定时， $B \propto (1 - p) / p$ ：miss 越少，预取时间越短。

当前 data：总长均为 32K = 32,768 token

miss 数 (NQL)	miss 比例	计算 C (ms / 层)	读取 D (MiB / 层)	需求 D/C (GiB/s)
128	0.3906%	1.178	43.828	36.333
512	1.5625%	3.702	43.312	11.425
2048	6.2500%	14.369	41.250	2.803
4096	12.5000%	28.593	38.500	1.315

看第一行与第三行：计算从 14.369 ms 缩到 1.178 ms；读取量仍为约 41 ~ 44 MiB。因此需求速率从 2.803 升到 36.333 GiB/s。

“正比”是近似，不是 data 的硬规则：miss 数翻 4 倍时，128→512 的计算时间只变为约 3.14 倍。上下文长度、固定开销、批处理与执行效率也会影响计算成本。

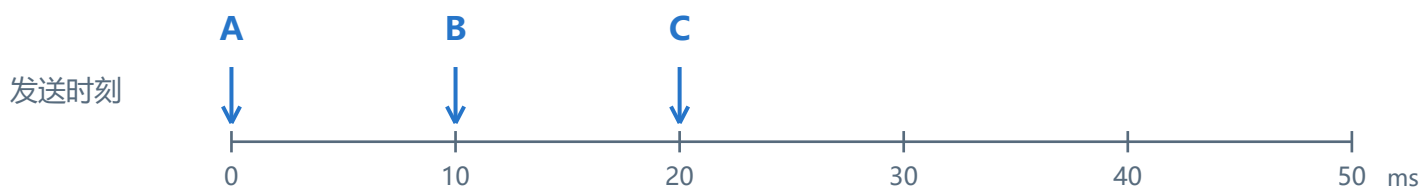
D 取自 Data（数据）；旧版用 V 表示 Volume（数据量），并非 KV 中的 Value。数值来自 data，C 与 D 分别输入；1 GiB = 1,024 MiB。来源 [2][3]。

开环与闭环：前面慢了，还继续发吗？

这一页只讨论：外部客户端把完整请求送进 NPU 等待队列。先不考虑 I/O 和预取。

有 A、B、C 三个请求，原本每个处理 10 ms。现在 **A 变慢，需要 30 ms**，B 和 C 仍需 10 ms。“**发送**”是进入队列，不是开始执行。

开环：按闹钟，每隔 10 ms 发一个



B 在 10 ms 到达，C 在 20 ms 到达。A 还没做完，B、C 就先排队。**A 变慢，不改变后面的发送时间。**

闭环：这个做完，再发下一个



等 A 在 30 ms 完成才发 B；等 B 在 40 ms 完成才发 C。**前面处理变慢，后面的发送跟着推迟。**

“有限”只表示清单会用完。

开环有限：按时间表发完这三个，再等它们结束。

闭环有限：做完一个发一个，第三个完成后结束。

此处用“一个客户端、每次只等待一条未完成请求”说明闭环。闭环也可以有多个客户端并发；它不意味着全系统一次只能执行一个请求。
[4]

先分清：到底是谁在发什么？

完整请求进入等待队列，与 NPU 为某一层发起读取，是两个不同位置的事件。

位置一：外部输入 → 每张 NPU 的等待队列

之前的实验先生成有限清单，所有请求在 $t = 0$ 到达。每张卡已有自己的输入顺序，后面变慢不会改写这个到达时间。

执行与排队：

当前请求

下一请求

后续请求

“接纳”表示从等待队列取出请求，开始占用本卡的执行位置。本组实验每卡同时计算一条请求；完整请求有多层，不会在到达时发完所有层的 I/O。

位置二：活跃请求 → 各层读取与计算

本层数据到齐，才能开始计算；**本层实际开算时，激活下一层预取**。因此一次读取等待，会推迟后续计算和后续读取。

本层数据迟到

本层计算推迟

下一层预取推迟

后面把这条因果关系称为“**内部计算—预取反馈**”。它解释了为什么有人把内部推进称为“闭环”，但不能据此认定外部也在等待完成后才发请求。

Baseline 和 Once per layer 共用这套逐层推进机制。不能用“一个开环、一个闭环”来解释两种策略的利用率差异。

源码事实：[3]。外部到达、每卡接纳与逐层预取分别见 6672、3156、3269 行。跨请求首层预取也受“下一请求已到达”限制。

NPU 为什么会停下来等？

计算当前层时，提前读取下一层；下一层的数据到齐，才可以继续算。

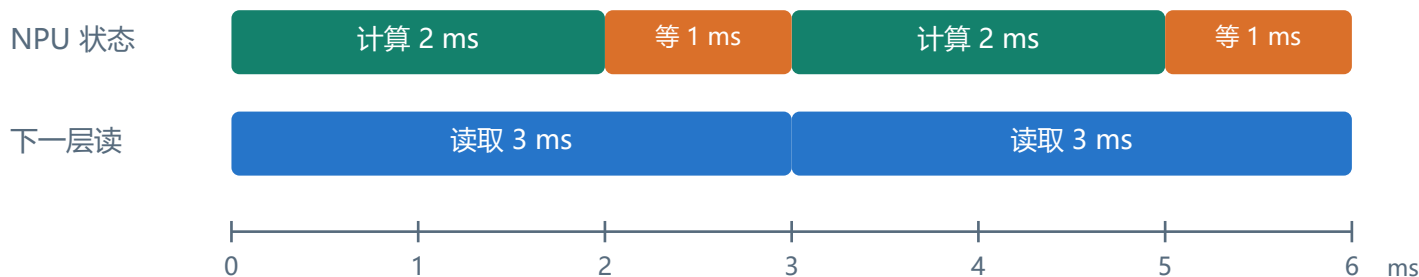
预取 = 提前搬下一层的数据。**I/O stall** = 算完了，但下一层数据还没到齐，只能等。

教学算例：每层计算 $C = 2\text{ ms}$ ；忽略启动与结束

A 读取只要 1 ms：全部藏在计算里



B 读取要 3 ms：每轮会露出 1 ms 等待



NPU 利用率 = 正在计算的时间 ÷ 观察时间

A: $6 / 6 = 100\%$ B: $4 / 6 \approx 66.7\%$

相同 C 和 D、稳定服务速率 b、一层预取的理想情况下：

$U \approx C / \max(C, D/b)$ 。U 是利用率；max 表示取两者较大值。

当前代码在本层开算时激活下一层读取；最后一层可预取已到达的下一请求首层。跨请求的预取预算来自前请求末层。来源 [3]。

外部开闭环，一定会改变利用率吗？

不一定。真正进入利用率公式的是计算时间；到达规则只有改变了执行或等待，才会影响它。

当前统计：32 张卡 × 2 秒 = 64 卡秒。

平均利用率 = 这 64 卡秒里，真正用于计算的时间 / 64 卡秒。若整窗只有计算与 I/O 等待，也可写成 $1 - \text{总等待} / 64 \text{ 卡秒}$ 。

教学例子：实际执行相同，就能得到相同利用率

一次全部放好

计算：57.6 卡秒

过程中及时补足

计算：57.6 卡秒



橙色均为 6.4 卡秒等待；两者利用率均为 90%

这个等价结论有条件：同一策略、相同初始执行状态；每卡任务和顺序相同；整个窗口都有任务；每次预取时下一请求已到；到达事件本身没有额外改变调度决策。此时外部到达方式可以不变计算和等待。

但严格“做完才发下一请求”可能打破这些条件。

当前模型会在前请求末层开算时，预取已到达的下一请求首层。若等前请求完成才发送，就失去了这段重叠机会。

因此，“队列里一直有活”还不够；也要保证**该预取时，下一个活已经可见**。

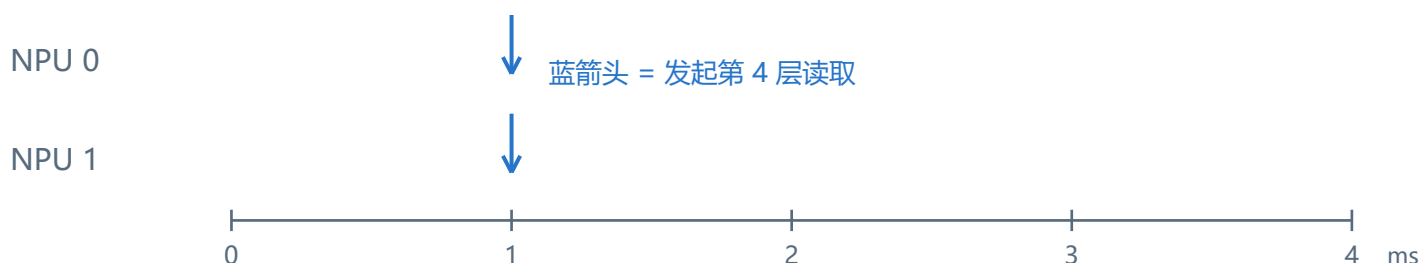
时间条为汇总卡时间的教学示意，不是单卡的 64 秒轨迹。只讨论利用率等价；不同到达时间仍可能改变从到达开始计时的延迟。源码 [3]。

自然错开：发生在不同 NPU 之间

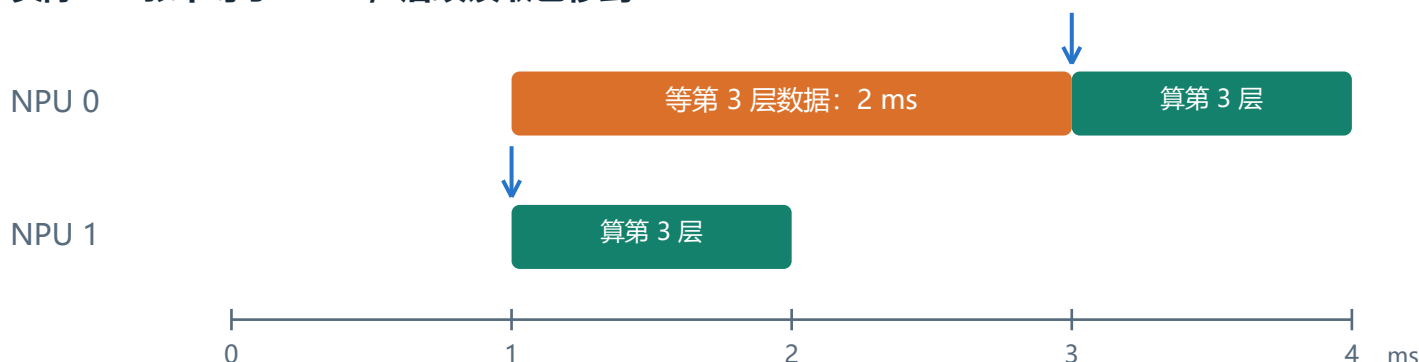
改变的是并发请求各层读取的相对时刻；没有重新排列各卡的输入队列。

教学例子：两卡原本都预计在 **1 ms 开始第 3 层计算**，同时发起第 4 层读取。现在 NPU 0 的第 3 层数据晚到；NPU 1 的进度没有改变。

原计划：第 4 层读取同时发起



实际：一张卡等了 2 ms，后续读取也移到 3 ms



两张卡的第 4 层读取，现在分别在 3 ms 与 1 ms 发起。单卡的等待改变了跨卡的相对时刻；计算时间不同、后续任务变化也会造成偏移。

错开不保证消除等待。不同时间入队的短读，仍可能遇到同一批未完成的长读取。后面是否再次聚集，需要继续推演。若仍把 NPU 0 的后续读取固定在 1 ms，分析的就不是当前反馈机制。

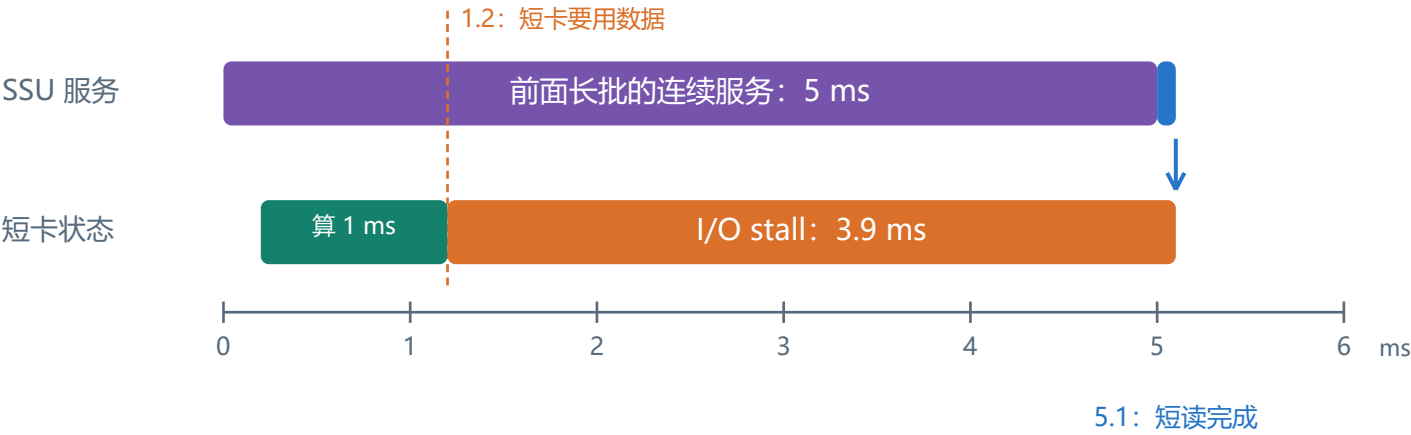
只画第 3 层计算和第 4 层读取的发起时刻，后续层省略。两卡不必处于相同层号才会竞争。当前计算起点触发后续预取：[3]。

一条 FIFO 队列，短读取怎样被挡住？

教学算例：只看 1 块 SSU 和 2 张 NPU，盘速率 40 GiB/s；忽略接收链路。

下一层读取	读取量 D	当前层计算 C	名义需求 D/C
长批读取	0.200 GiB	100 ms	2 GiB/s
短批读取	0.004 GiB	1 ms	4 GiB/s

t = 0: 长卡开算，并已把下一层的长批数据排进 FIFO。
t = 0.2 ms: 短卡开算并发出下一层读取；它希望在 **1.2 ms** 前拿到数据。



短读自身只需 0.1 ms，却等到 5.1 ms 才完成。
直接挡住它的是前面的读取工作；其他卡的长计算本身不占用这张短卡。

两卡名义需求合计 $2 + 4 = 6 \text{ GiB/s} < 40 \text{ GiB/s}$ ，仍可能发生这次等待。原因是数据成批排队，而短卡要求很快到齐。**D/C 欠载，不等于禁止瞬时突发。**

假设：长批已先入队，短读不能越过它。图中长条代表许多小 I/O 的累计服务，不是一条不可中断 5 ms 的命令。本例不能证明 32 卡长期下降 10%。

看带宽时，要分清三个数字

想验证“欠载下 FIFO 很差”，需要证明输入有余量，同时追踪数据是否按时到齐。

数字	它回答什么问题？
容量：每盘 40 GiB/s	这块盘最多能搬多快？6 块合计 240 GiB/s。
名义需求：D/C	如果希望读数藏在计算里，这个请求需要多快？
实际吞吐：已搬字节 / 时间	在这段时间里，盘实际上搬了多少数据？

实际吞吐低，不能独自证明输入欠载。
NPU 等待 → 后面的计算推迟 → 后续读取也推迟 → 盘收到的工作可能减少。这里说的是内部计算—预取反馈，与外部是否按闹钟发送是两回事。

逐盘检查：此前名义口径是对每块盘，累加**当前已接纳请求**的 D_s/C (D_s 是该盘读取量)，而非只与总容量 240 比较。跨请求首层预取的额外释放及前请求末层预算应另列；名义欠载不能保证每次读取都按时完成。

一个反例：第 2 页的 32K / NQL=128，每卡需求约 36.333 GiB/s。若 32 张卡同时运行这种画像，合计约 **1,162.65 GiB/s**，已远超 240。这种情况下利用率低，不能作为“欠载 FIFO 缺陷”的证据。

因此，要同时看 **D（读取批量）、C（可等多久）、逐盘分布、请求顺序和释放时刻**。只说“有长流和短流”，条件还不够。

容量取自此前 32 NPU / 6 SSU 实验配置；每卡接收链路为 50 GiB/s，也会影响到齐时间。数据算术见 [2]；闭环行为见 [3][4]。

有限请求：窗口选法也会改变答案

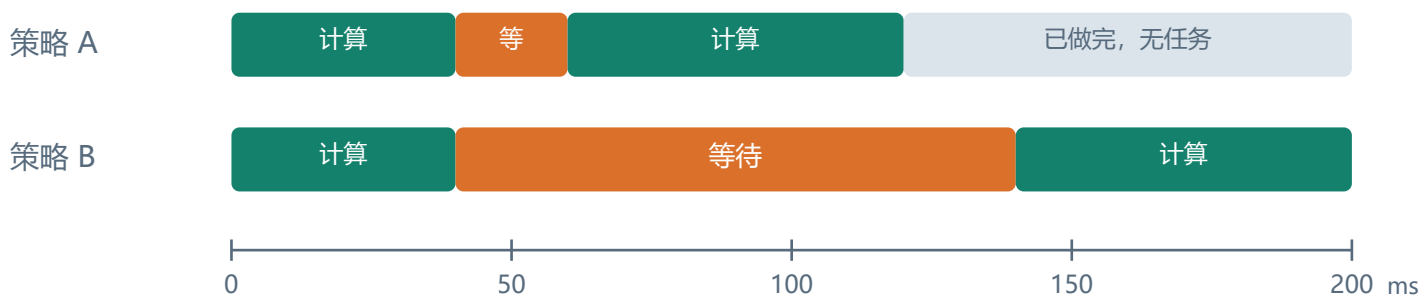
warm 是先让系统运行一段时间再统计；它本身不保证已经进入稳定状态。

之前的主统计窗口是 [2 s, 4 s)，共 2 秒。

32 卡平均利用率 = 窗口内 32 卡计算时间之和 / 64 卡秒

每张卡都有任务时，剩下的时间主要体现等待；队列耗尽后，还会混入无任务空闲。

教学例子：同一张卡、同一批任务，总计算都是 100 ms



都统计 0 ~ 200 ms：A 与 B 都是 $100 / 200 = 50\%$ 。

但完成时间不同：A 在 120 ms 做完，B 在 200 ms 做完。按各自完成时间算，分别是 83.3% 和 50%。

所以，扩大窗口时要同时报告：每卡是否还有任务、I/O 等待多少、总完成时间。不同卡的层读取时刻也可能逐渐分散；一次突发造成等待，不代表以后永远这样。

miss 改变后，也不要只比利用率。命中更多，真正需要的计算减少，即便利用率变低，请求仍可能更早完成。判断策略时应固定输入，并一起看请求延迟、SLO 达标率与吞吐。

本页时间线为教学算例。之前的 TTFT SLO 口径是“接纳后到完成”，不包含接纳前排队；不能直接等同于用户从到达开始等待首 token 的时间。

读分析时，用这张对照表

先核实假设，再看公式；单次现象、长期结论与真实实验结果需要分开。

看到的说法	应该怎样理解
miss 长度 / miss 比例	长度 $M = pL$ ；同为 10% miss，总长不同，计算量也不同。
计算正比 miss 长度	固定条件下的近似；实际 C 查 data，不强行套同一常数。
带宽决定利用率	稳定服务的简化模型可估算；FIFO 还要看数据到齐时间。
开环 / 闭环	先区分外部请求发送、内部层预取反馈；不能据标签判断 U 高低。
有限请求	任务清单会耗尽；扩大窗口可能混入结束后的空闲。

比较 Baseline 与 Once，应固定输入和逐层推进机制。研究它们的路径分配与服务顺序，怎样改变各卡的数据到齐时刻、等待和利用率。

核实来源（点击可打开）

[1] vLLM: Automatic Prefix Caching

命中前缀可复用 KV，跳过该部分的重复计算。

[2] 本项目 data 与 token 划分

第 2 页取 data 的 32K 四行；sim.py:231 定义计算与读取 token。

[3] 本项目 continuous_batch_sim.py

784 行读取 C；3156 行接纳；3269 行逐层预取；6672 行外部到达。

[4] Grafana k6: Open and closed models

开环与闭环到达模型、系统变慢对闭环发送速率的影响。

核对日期：2026-09-10。源码版本：e56b1f95bca6。本次仅核对代码与 data、制作教学图，没有重跑仿真。